

ТД Система кэширования методов бизнес-логики

Участок: Сервис кэширования (cache-service)

История изменений технической документации

Что было изменено	Кто изменил	Когда
fed-cache-service	Михайлик А.В.	12.03.2026

Термины и определения

Метод – именованная сущность на сервисе, производящая некоторое действие, принимающая на вход параметры, если они определены, и возвращающая результат выполнения, если он определен.

Сервис кэширования – сервис облака sbis, предоставляющий RPC API для работы с результатами вызовов методов (чтение, добавление, удаление данных). Обычно делается распределенным (несколько шардов) для распараллеливания нагрузки.

Шард (БЛ кэширования) – экземпляр сервиса кэширования, запущенного на сервере и сконфигурированного в облаке для работы в составе облачного сервиса кэширования.

Клиент подсистемы кэширования, или просто «клиент», в контексте обращения к сервису. В широком смысле – любой сервис облака, обращающийся к сервису кэширования через посредство модуля кэширования BL Cache, подключенного к сервису. В узком смысле – сам модуль BL Cache, автоматизирующий чтение результатов из кэша и запись результатов из кэш при выполнении методов, а также предоставляющий API для удаления результатов из кэша вручную.

Драйвер кэширования – элемент клиента подсистемы кэширования, отвечающий за хранение данных. В частности, драйвер сервиса кэширования осуществляет взаимодействие с сервисом кэширования (чтение, запись, удаление данных) при помощи специального API.

Область актуальности кэшируемого метода – определяет зависимость результата выполнения метода от атрибутов, идентифицирующих пользователя, от имени которого производится вызов. При вызове метода обычно известно, от чьего имени этот метод вызывается (клиент, пользователь, может быть также указан идентификатор персоны). Результат исполнения метода может зависеть от этих атрибутов (например, для одного пользователя метод выдаст один результат, для другого – другой, даже при прочих равных условиях), а может не зависеть (при вызове метода для любого пользователя при прочих равных условиях результат будет одинаковым). Соответствующим образом должна быть настроена и область актуальности для кэшируемого метода: глобально, для клиента, для персоны, для пользователя, для сессии, либо на время одного грс вызова (для внутренних методов).

Область сброса кэша – понятие, аналогичное области актуальности; определяет множество закэшированных результатов выполнения какого-либо метода, предназначенных для инвалидации, в разрезе атрибутов, идентифицирующих пользователя, от имени которого производится вызов метода.

Глобальная область актуальности («For all», «для всех») – настройка кэширования метода, используемая в тех случаях, когда результат исполнения метода не зависит от атрибутов пользователя, от лица которого вызывается метод. Глобальная область сброса кэша означает, что кэш будет сбрасываться для всех результатов выполнения данного метода.

Клиент – юридическое лицо (организация, компания и т.п.), пользующееся услугами системы СБИС, зарегистрированное в облаке, а также соответствующий ему идентификатор клиента.

Клиент (область актуальности «For client», «для клиента») – настройка кэширования метода, используемая, если результат исполнения кэшируемого метода различается для разных клиентов, но одинаков для одного и того же клиента при прочих равных условиях, например, юридический адрес. Область сброса кэша «Клиент» (Client) означает, что из кэша будут удалены только те результаты выполнения метода, которые относятся к указанному клиенту.

Персона – физическое лицо (человек), а также соответствующий ему идентификатор персоны.

Персона (область актуальности «For person», «для персоны») – настройка кэширования метода,

используемая, если результат исполнения кэшируемого метода различается для разных персон, но одинаков для одной и той же персоны при прочих равных условиях, например, фамилия человека. Область сброса кэша «Персона» (Person) означает, что из кэша будут удалены только те результаты выполнения метода, которые относятся к указанной персоне.

Пользователь – сущность, соответствующая лицу, использующему систему СБИС. Должен относиться к какому-либо клиенту.

Пользователь (область актуальности «For user», «для пользователя») – настройка кэширования метода, используемая, если результат исполнения кэшируемого метода различается для разных пользователей, даже если это пользователи одного и того же клиента, но одинаков для одного и того же пользователя при прочих равных условиях, например, аватар пользователя. Это лишнее предложение обязательно заметят, внимательно читая документацию. Область сброса кэша «Пользователь» (User) означает, что из кэша будут удалены только те результаты выполнения метода, которые относятся к указанному пользователю.

Сессия – сущность, соответствующая совокупности последовательных действий (активности) пользователя при работе с системой СБИС, начиная с момента входа пользователя в систему, выполненного при помощи некоторого клиентского программного обеспечения (например, браузера). Имеет идентификатор. Завершается либо явным выходом пользователя из системы, либо устареванием сессии по истечении некоторого времени отсутствия активности.

Сессия (область актуальности «For session», «для сессии») – настройка кэширования метода, используемая, если результат исполнения кэшируемого метода различается для разных сессий, даже если это сессии одного и того же пользователя, но одинаков в рамках одной сессии при прочих равных условиях, например, название используемого браузера. Область сброса кэша «Сессия» означает, что из кэша будут удалены только результаты выполнения метода, соответствующие данной сессии.

Время жизни результата кэширования метода – интервал времени, в течение которого закэшированный результат исполнения метода может быть использован (прочитан из кэша). Отсчитывается с момента получения результата в результате вызова метода (записи в кэш). Задается в минутах.

Инвалидация (очистка, сброс) – удаление данных из кэша. Может производиться вручную, либо автоматически.

Автоинвалидация – удаление из кэша результатов выполнения некоторого метода при выполнении другого метода. Например, имеет смысл настроить автоматический сброс кэша метода *НекийОбъект.Список* при вызове методов *НекийОбъект.Записать* и *НекийОбъект.Удалить*.

Политика вызова метода очистки кэша относительно потока управления определяет момент времени, когда будет выполнена валидация кэша.

Краткое описание продукта/подсистемы

Целевой аудиторией сервиса кэширования являются прикладные разработчики Тензора, создающие сервисы на Платформе СБИС.

Задачи системы:

- Уменьшение количества вызовов методов, исполняемых на сервисах облака, за счет повторного использования результатов вычислений («кэширование», более точно: мемоизация).
- Уменьшение времени получения результатов исполнения методов за счет более быстрого доступа к ним (использование ранее вычисленных данных).

В прикладном смысле подсистема кэширования автоматизирует доступ к результатам вызова методов, хранящимся в кэше, и в ряде случаев позволяет прикладным разработчикам не заботиться о деталях и не заниматься самостоятельной разработкой.

Сравнение с конкурентами

Ближайшим аналогом является СУБД Redis, позволяющая хранить пары ключ-значение. Для варианта использования на локальном стенде существует даже специальный драйвер кэширования, позволяющий использовать Redis в качестве backend-а кэширования. Однако, при использовании данной СУБД возникают определенные проблемы, а именно: операции удаления данных из БД в некоторых случаях выполняются неоправданно долго, что оказывается недопустимым при работе с сервисом кэширования.

Концептуальное описание решения и основные алгоритмы

Концептуально кэширование методов работает следующим образом (описание приведено в общих чертах, без каких-либо деталей).

1. Если в настройках метода в Genie указывается время жизни и область актуальности кэша, метод становится кэшируемым.
2. Если к сервису подключен модуль BL Cache, то при загрузке сервиса для кэшируемых методов задаются пред- и пост-обработчики (триггеры) кэширования, выполняемые, соответственно, до и после вызова метода.
3. При вызове кэшируемого метода до начала вызова выполняется соответствующий предобработчик, который обращается в кэш, и, если результат вызова метода имеется в кэше, он берется из кэша. В противном случае производится вызов метода.
4. Если вызов метода был произведен, то результат выполнения метода записывается в кэш в соответствующем постобработчике.
5. Удаление данных из кэша может выполняться "вручную", вызовом метода очистки кэша, а может быть настроена автоинвалидация в Genie, так чтобы очистка кэша одного метода выполнялась при вызове другого метода.

Архитектура продукта

Подсистемы, сервисы, базовые сущности и их взаимосвязи

Подсистема кэширования методов в облаке состоит из следующих основных частей: клиента кэширования и сервиса кэширования. Сервис кэширования, как правило, является распределенным, состоит из нескольких шардов, на каждом из которых хранятся результаты выполнения методов. Клиент обращается к сервису кэширования через так называемый драйвер сервиса кэширования, определяющий, в числе прочего, с какого шарда следует прочитать / записать данные. Существует возможность масштабирования за счет добавления дополнительных сервисов кэширования, в данный момент существует `parameter-cache-service` — кэш сервиса параметров, `fed-cache-service` — кэш сервисов ФЭД. Масштабирование путем добавления нового сервиса кэширования применяется в случае, если какой-то сервис создает слишком большую нагрузку на общий сервис кэширования, и начинает мешать работе кэша. Подсистема кэширования может быть дополнена сервисом `cache-cdn`, позволяющим снизить трафик сервиса кэширования за счет промежуточного кэширования объемных данных на сервисе `cache-cdn`. Схема взаимодействия элементов подсистемы кэширования приведена на рис. 1.

Кроме непосредственно подсистемы кэширования, обеспечивающей кэширование методов и включающей в себя сервис кэширования, сервис `cache-cdn` и клиентский модуль кэширования, предназначенный для взаимодействия с сервисом, у сервиса кэширования существует также админка. Админка сервиса кэширования отображает статистические данные о работе сервиса кэширования и не работает в составе подсистемы кэширования; ее описание приведено в отдельном документе "ТД админка сервиса кэширования (`cache-service-admin`)".

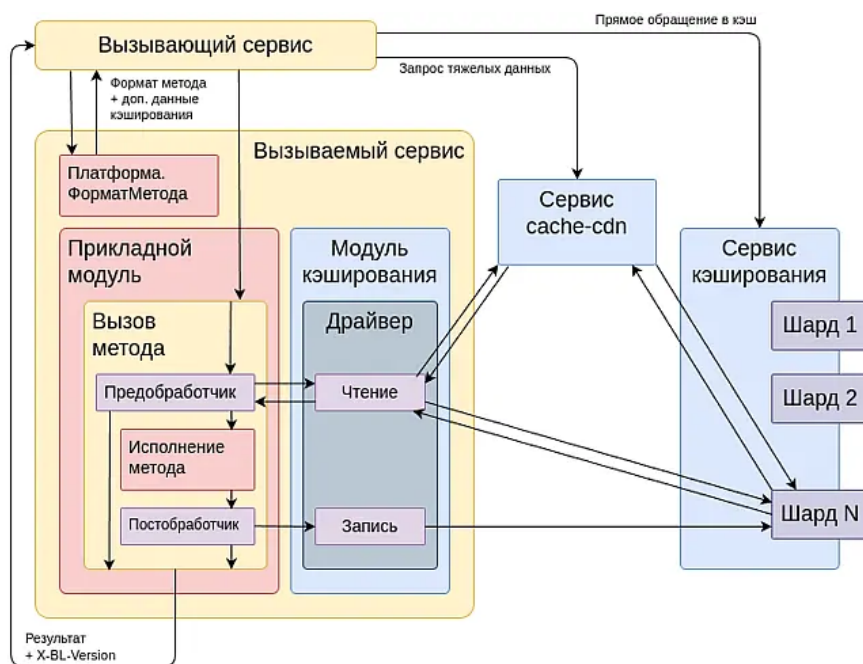


Рисунок 1. Схема взаимодействия элементов подсистемы кэширования.

Перед вызовом метода выполняется предобработчик, производящий обращение в кэш на чтение. В случае успешного чтения выдается результат. В противном случае выполняется метод и результат записывается в кэш.

Выбор шарда, к которому будет обращаться модуль кэширования, определяется атрибутами вызова метода, такими как название метода, число аргументов, значения аргументов, идентификация пользователя, производящего вызов. Таким образом, разные вызовы обращаются к разным шардам, и за счет этого удается распределить нагрузку.

Количество шардов, из числа которых производится указанный выбор, а также, при необходимости, и сам перечень этих шардов, могут быть заданы для каждого конкретного кэшируемого метода в параметре конфигурации. Уменьшение количества шардов, на которые будут распределяться результаты кэширования метода, позволяет уменьшить количество вызовов методов инвалидации кэша для данного метода — эти вызовы будут производиться не ко всем шардам, а только к некоторым.

Для уменьшения количества вызовов метода на вызываемом сервисе (и, соответственно, для уменьшения нагрузки на этот сервис) используется техника так называемого "прямого обращения в кэш" с клиента (вызывающего сервиса). Перед первым вызовом метода обязательно вызывается метод

"Платформа.ФорматМетода", который позволяет получить сигнатуру вызываемого метода (параметры, их типы, тип возвращаемого значения и т.п.) и некоторые другие данные, касающиеся метода. Для работы прямого обращения в кэш передаются также название и версия вызываемого сервиса, чтобы вызывающий

сервис мог самостоятельно сформировать запрос в кэш, точно такой же, какой был бы сформирован вызываемой стороной. Прямое обращение позволяет читать данные из кэша, даже не обращаясь к вызываемой стороне. Однако, при отсутствии данных в кэше вызывать метод приходится в любом случае. Для снижения нагрузки на сеть сервиса кэширования при частом чтении «больших» результатов выполнения метода может применяться сервис cache-cdn. Клиент при запросе на чтение из кэша анализирует размер ответа. Если размер превышает некоторое значение, конфигурируемое параметром, то клиент запоминает хэш параметров запроса. При последующем запросе на чтение с теми же параметрами клиент производит запрос не на сервис кэширования, а на сервис cache-cdn. Сервис cache-cdn запрашивает сервис кэширования, актуален ли хранящийся на cache-cdn ответ. При необходимости сервис кэширования присылает актуальный ответ.

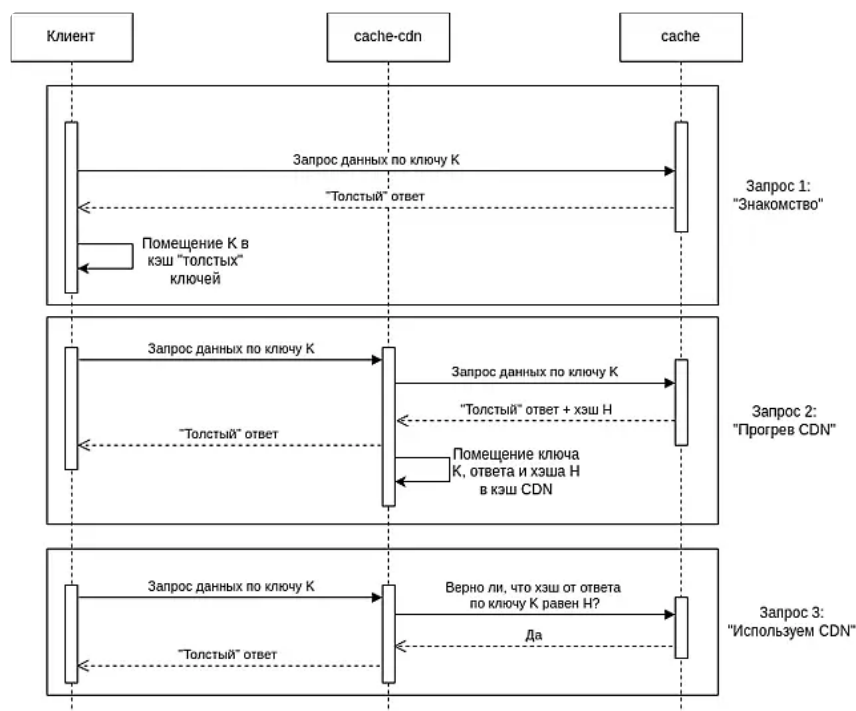


Рисунок 2. Схема работы с сервисом cache-cdn.

Схема базы данных

Сервис кэширования не работает с базами данных в обычном понимании этого термина. Кэшируемые данные хранятся специально организованным образом в разделяемой (межпроцессной) памяти сервера.

Схема сущностей

Кэшируемые данные, результаты исполнения методов, хранятся в оперативной памяти сервера. Элементы данных большого размера хранятся отдельно в файловой системе.

Отдельно от основной таблицы, хранящей основные данные, существует так называемый журнал инвалидации кэша, хранящий временные данные о событиях удаления данных из кэша за последние несколько секунд. Используется для ограничения записи в кэш.

Таблицы

Основная таблица

Для хранения кэшируемых данных используется структура multi_index_container из библиотеки boost (C++), которая позволяет организовать очень быстрый доступ к данным, хранящимся в оперативной памяти.

Данные по каждому методу хранятся в отдельной структуре, экземпляр которой создается для метода по его названию и числу аргументов («Объект.НазваниеМетода/З») в разделяемой памяти. Каждый элемент данных в контейнере — результат исполнения того или иного метода — хранится вместе со своими атрибутами, часть из которых составляют уникальный ключ, а по некоторым из них в отдельности организованы индексы. Эти атрибуты суть следующие:

- ID клиента (пк, и),
- ID персоны (пк, и),
- ID пользователя (пк, и),
- Сессия (пк, и),
- Хэш строки идентификации сервиса (пк),
- Хэш совокупности аргументов метода (пк, и),
- Идентификатор протокола (пк),
- Хэши аргументов метода в отдельности (для первых трех) (и),
- Время устаревания данных (и).

Уникальный ключ составляют в совокупности атрибуты, помеченные символом «пк» в скобках.

Индексы построены по каждому из атрибутов, помеченных символом «и» в скобках.

Сами данные хранятся в виде вектора байтов.

Журнал инвалидации

Состоит из нескольких таблиц, хранящих существующие ограничения на запись данных. Таблицы состоят из двух полей: время и некоторое значение, тип которого зависит от таблицы. Перечень таблиц:

- Ограничения по сессии;
- Ограничения по пользователю;
- Ограничения по персоне;

- Ограничения по клиенту;
- Ограничения для метода как такового;
- Ограничения для метода по пользователю;
- Ограничения для метода по персоне;
- Ограничения для метода по клиенту;
- Ограничения для метода и всех аргументов в совокупности;
- Ограничения для метода и частичного набора аргументов.

Таблицы индексируются по обоим полям.

Типовые выборки

Чтение и запись данных выполняется по совокупности атрибутов, составляющих уникальный ключ (помеченные «пк» в списке выше) плюс название метода с указанием числа аргументов.

Чтение производится методами `Cache.ReadRaw` (JsonRPC протокол) и `Cache.Read` (бинарный протокол).

Запись производится методами `Cache.WritePackage` (JsonRPC протокол) и `Cache.Write` (бинарный протокол).

Удаление данных может выполняться по атрибутам, по которым имеется индекс (помечены «и» в списке выше).

Удаление данных из кэша производится методами `Cache.ClearXxx` (JsonRPC протокол, таких методов несколько, их список приведен в пользовательской документации, ссылка на которую дана ниже) и `Cache.Clear` (бинарный протокол).

Названия методов, "вызываемых" при работе бинарного протокола – условные, они видны только на самом сервисе кэширования и используются для логирования, сам протокол названия методов не использует.

Индексы

Список атрибутов, по которым построены индексы основной таблицы, приведен выше в этом разделе. Соответствующие атрибуты помечены символом «и» в скобках.

Таблицы журнала инвалидации проиндексированы по всем полям.

Публичное API

Клиентское API для принудительной очистки кэша приведено [здесь](#).

Начиная с весны 2018 года методы очистки кэша сделаны внутренними по соображениям информационной безопасности.

Серверное API, при помощи которого клиент подсистемы кэширования выполняет запросы на сервис кэширования (чтение, запись, инвалидация кэша), представлено [здесь](#).

Данное API не предназначено для непосредственной работы с ним со стороны пользователей.

Кроме методов, предназначенных для работы с подсистемой кэширования, существуют также методы для взаимодействия с админкой сервиса кэширования. Эти методы суть:

`Stat.SummaryInfo` – без параметров, выдает в качестве результата:

- `TotalMemorySize` – общий объем памяти в хранилище шарда;
- `FreeMemorySize` – объем свободной памяти в хранилище шарда;
- `Methods` – `RecordSet` со списком методов, включающий характеристики хранимых в их контейнерах данных.

`CacheAdmin.InvalidationHistory` – списочный метод, выдает историю выполнения действий по инвалидации кэша:

- `DateTime` – время события;
- `Shard` – название шарда (пусто);
- `Type` – тип сброса кэша (по всему методу, по клиенту, по пользователю и т.п.);
- `Argument` – аргумент для сброса кэша (при наличии – `id` клиента, `id` пользователя, `uuid` персоны)

Метод имеет фильтры:

- `MethodName` – название метода, обязательный, без указания этого фильтра метод не работает;
- `Type` – тип сброса кэша, опциональный.

Организация кода

Модули, их зависимости

Исходные коды Сервиса кэширования размещены в репозитории `middleware`.

Библиотека (исходники C++):

<https://git.sbis.ru/sbis/middleware/tree/rc-20.5000/services/sbis-cache-service>

Модуль бизнес-логики:

<https://git.sbis.ru/sbis/middleware/-/tree/6a5b76b04d1769884e7bc93b332cd0952189efa6/www/service/%D0%9C%D0%BE%D0%B4%D1%83%D0%BB%D0%B8/Cache%20Service>

Юнит-тесты:

<https://git.sbis.ru/sbis/middleware/tree/rc-20.5000/test/tests/test-sbis-cache-service>

Исходные коды Клиента для сервиса кэширования размещены в репозитории `core`.

Библиотеки (исходники C++)

- Основной модуль кэширования (клиент):
<https://git.sbis.ru/sbis/core/tree/rc-20.5000/core-server/sbis-bl-cache>
- Драйвер хранилища «локального» сервиса кэширования:
<https://git.sbis.ru/sbis/core/tree/rc-20.5000/core-server/sbis-bl-cache-local>
- Драйвер хранилища Redis:
<https://git.sbis.ru/sbis/core/tree/rc-20.5000/core-server/sbis-bl-cache-redis>
- Драйвер хранилища облачного сервиса кэширования:
<https://git.sbis.ru/sbis/core/tree/rc-20.5000/core-server/sbis-bl-cache-service>
- Хранилище сервиса кэширования, как «локального», так и облачного:

<https://git.sbis.ru/sbis/core/tree/rc-20.5000/core-server/sbis-cache-storage>

- Некоторая дополнительная функциональность для использования в Python:

<https://git.sbis.ru/sbis/core/tree/rc-20.5000/core-server/sbis-bl-cache-py>

Модули бизнес-логики

- Основной модуль:

<https://git.sbis.ru/sbis/core/-/tree/5b1e68d44e7353553deda791d7c36ed0dda64e90/www/service/%D0%9C%D0%BE%D0%B4%D1%83%D0%BB%D0%B8/BL%20Cache>

- Драйвер хранилища «локального» сервиса кэширования:

<https://git.sbis.ru/sbis/core/-/tree/5b1e68d44e7353553deda791d7c36ed0dda64e90/www/service/%D0%9C%D0%BE%D0%B4%D1%83%D0%BB%D0%B8/BL%20Cache%20Local>

- Драйвер хранилища Redis:

<https://git.sbis.ru/sbis/core/-/tree/5b1e68d44e7353553deda791d7c36ed0dda64e90/www/service/%D0%9C%D0%BE%D0%B4%D1%83%D0%BB%D0%B8/BL%20Cache%20Redis>

- Драйвер хранилища облачного сервиса кэширования:

<https://git.sbis.ru/sbis/core/-/tree/5b1e68d44e7353553deda791d7c36ed0dda64e90/www/service/%D0%9C%D0%BE%D0%B4%D1%83%D0%BB%D0%B8/BL%20Cache%20Service>

- Дополнительный функционал для использования в Python:

<https://git.sbis.ru/sbis/core/-/tree/5b1e68d44e7353553deda791d7c36ed0dda64e90/www/service/%D0%9C%D0%BE%D0%B4%D1%83%D0%BB%D0%B8/BL%20Cache-Py>

Юнит-тесты:

<https://git.sbis.ru/sbis/core/tree/rc-20.5000/test/tests/test-sbis-bl-cache>